Kafka 이벤트 기반 시스템에서 숨은 장애와 유실을 잡은 과정

2026-10-13 · 최유준

---

1. 프롤로그

이벤트 기반 아키텍처를 도입할 때 우리는 흔히 **'Kafka를 쓰니까, Outbox 패턴을 적용했으니까 안전하겠지'**라고 생각합니다. 저 역시 컨슈머가 멈추면 울릴 알림까지 넣어 두고, 실제로 울리는지는 한 번도 확인하지 않았습니다.

하지만 브로커가 죽거나 서비스가 갑자기 다운되는 것처럼, 코드로는 막을 수 없는 장애가 나도 메시지는 단 한 건도 유실되지 않을까요? 멈췄다면 알림은 제대로 울릴까요? 울렸다면 어디서, 왜 멈췄는지 찾아갈 수 있을까요?

이 글에서는 제가 맡은 Game 서비스(운동 기록 이벤트를 받아 퀘스트를 판정하고 XP를 지급하는 서비스)에서 Kafka 브로커 3대 중 2대를 끄고, Outbox 발행기를 전송 도중 강제 종료하며 정말 유실이 0인지 확인했습니다. 그리고 시스템은 멈췄는데 지표는 정상이라고 말하던 모니터링의 허점을 지표 · 로그 · 트레이스 순서로 잡은 과정을 정리합니다.

---

2. 설계상 보장이 정말 유실을 없앨까?

2-1. 브로커 3대 중 2대가 죽으면

설계상 보장



Game이 쓰던 Kafka는 브로커가 한 대였습니다. 이 한 대가 디스크째 망가지면, 아직 아무도 읽지 않은 이벤트도 함께 사라집니다.

운영 서버는 메모리 때문에 한 대를 유지하고, 로컬에 3대 구성을 따로 만들어 실험했습니다.

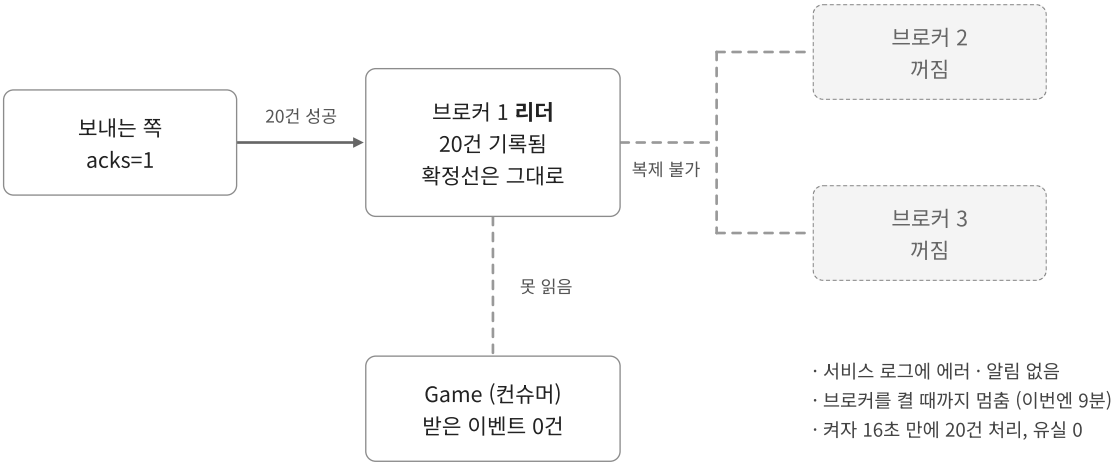
- 모든 메시지를 3대에 복제한다 (replication factor 3)
- 최소 2대에 기록돼야 쓰기 성공으로 친다 (min.insync.replicas=2)
- 복제가 덜 된 브로커는 리더가 될 수 없다 (unclean leader election 끔)

이론대로라면 1대가 죽어도 서비스는 계속되고, 2대가 죽으면 쓰기가 실패해야 합니다. 쓰기가 실패하면 보내는 쪽 Outbox에 미발행 메시지가 쌓이고, Outbox 알림이 울릴 거라고 예상했습니다.

실제 장애

실제 장애: 쓰기는 성공했는데 읽기가 멈췄다

브로커 2 · 3을 끄자 리더가 모두 브로커 1로 넘어갔고, 20건은 브로커 1에만 기록됐다



복제본이 min.insync.replicas(2)보다 적어 리더가 확정선(high watermark)을 올리지 않은 것으로 추정. 컨슈머는 확정선 아래만 읽는다.

1대는 예상대로였습니다. 끈 브로커가 리더이던 파티션은 리더가 바로 다른 브로커로 넘어갔고, 그동안 들어온 100건이 모두 처리됐습니다. 유실 0, 중복 지급 0이었습니다.

2대는 예상과 달랐습니다. 처음에 리더는 세 브로커에 고르게 나뉘어 있었고, 2대를 끄자 그 둘이 맡던 파티션의 리더가 모두 남은 한 대로 넘어갔습니다. 이 상태에서 이벤트 20건을 보냈는데, 쓰기는 실패하지 않았습니다. 실험 도구가 acks=1(리더에만 기록되면 성공)로 보냈기 때문입니다.

그런데 Game은 이 20건을 한 건도 받지 못했습니다. 서비스 로그에는 에러도 리밸런스도 없었고, 꺼진 브로커를 다시 켤 때까지(이번 실험에서는 9분) 그대로 멈춰 있었습니다. 브로커 한 대를 다시 켜자, 그 브로커가 20건을 복제해 간 뒤 16초 만에 모두 처리됐습니다. 유실과 중복은 0이었습니다.

원인은 이렇게 보고 있습니다. 컨슈머는 high watermark, 즉 충분히 복제됐다고 확정된 위치까지만 읽을 수 있습니다. 실험에 쓴 Kafka 4.3.1(ELR이 4.1부터 새 클러스터 기본값)에서는 살아 있는 복제본이 min.insync.replicas보다 적으면 리더가 이 선을 올리지 않습니다(ELR). 20건은 리더 한 곳에만 있었기 때문에 확정되지 못했고, 그래서 컨슈머에게 보이지 않았습니다. 토픽 상태의 ELR 표시, 서비스 로그에 오류가 없었던 점, 복제가 회복되자마자 처리된 점을 근거로 한 추정입니다.

참고: Apache Kafka 4.3 문서 · Eligible Leader Replicas

Kafka가 한 곳에만 있는 데이터라도 읽게 해 줬다면 어땠을까요? Game은 그 이벤트로 XP를 지급했을 것이고, 직후에 그 브로커의 디스크가 망가지면 XP는 나갔는데 근거가 된 이벤트는 어디에도 남지 않게 됩니다. Kafka는 이걸 막으려고 읽기까지 멈춘 것입니다.

실험: 실행 도구 · 결과 · 처리량 그래프(멈춤 구간)

결론

데이터는 잃지 않았고, 그 대가로 브로커가 돌아올 때까지 처리가 멈췄습니다. 가용성을 내주고 정합성을 지킨 셈입니다.

다만 이 유실 0은 Kafka 혼자 지킨 게 아닙니다. 실험 도구는 acks=1로 보내서 20건이 남은 한 대에만 있었고, 그 한 대마저 망가졌다면 진짜로 사라졌을 것입니다. 실제 Health는 acks=all로 보냅니다.

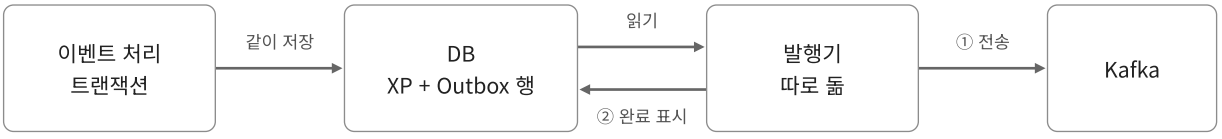
따라서 운영에서 같은 장애가 나면 20건은 브로커에 들어가지 않고 쓰기 실패로 돌아와, 보내는 쪽이 실패를 알 수 있습니다. Kafka 안에 한 대에만 있는 데이터가 생기지 않는 것입니다. 실패한 메시지를 다시 보내는 건 Health 쪽 몫입니다. 유실 0은 브로커 설정과 보내는 쪽의 acks 설정이 함께 만든 결과입니다.

2-2. 전송 도중 서비스가 다운되면

설계상 보장

설계상 보장: DB에 저장된 메시지는 언젠가 발행된다

비즈니스 데이터와 보낼 메시지를 한 트랜잭션으로 저장하고, 발행기가 따로 Kafka로 보낸 뒤 완료로 표시한다

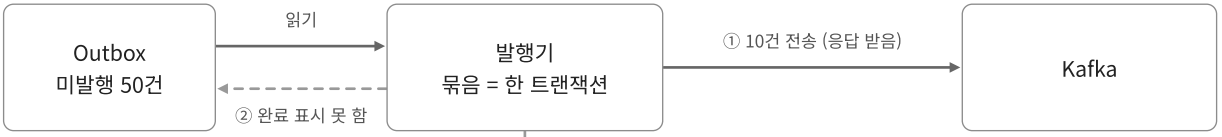


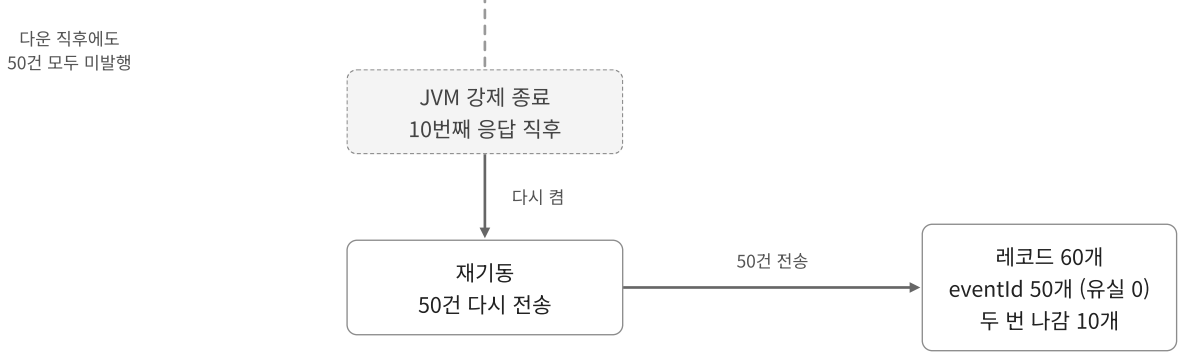
Outbox 패턴은 보낼 메시지를 비즈니스 데이터와 같은 트랜잭션으로 DB에 저장하고, 별도의 발행기가 Kafka로 보낸 뒤 **발행 완료**로 표시하는 방식입니다. DB에 저장된 이상 메시지는 사라지지 않고, 언젠가는 발행됩니다.

실제 장애

실제 장애: 전송과 발행 완료 커밋 사이에 서비스가 다운되면 중복 발행된다

발행기는 최대 100건을 한 트랜잭션으로 보내고, 완료 표시는 묶음이 끝날 때 한꺼번에 커밋한다





발행은 **최소 한 번(at-least-once)**. 중복 폭은 묶음 안에서 다운되기 전까지 보낸 건수(최대 100)이고, 같은 eventId라 받는 쪽이 걸러 낼 수 있다.

그런데 Kafka로 보내는 것과 DB에 완료를 표시하는 것은 서로 다른 시스템에 쓰는 두 동작이라 하나로 묶을 수 없습니다. Outbox가 서비스 쪽에서 없앤 이중 쓰기 문제가 발행기 쪽에 그대로 남아 있는 것입니다. 전송과 발행 완료 커밋 사이에 서비스가 다운되면 같은 메시지가 다시 나갑니다.

이 틈이 얼마나 넓은지는 코드를 보고 예측했습니다. 발행기는 미발행 메시지를 최대 100건씩 가져와 한 트랜잭션에서 보내고, 완료 표시는 그 묶음이 끝날 때 한꺼번에 커밋합니다. 그렇다면 중복은 1건이 아니라 **다운되기 전까지 보낸 건수**만큼 생길 것입니다.

실험으로 확인했습니다. 발행을 막아 둔 채 미발행 메시지 50건을 쌓고, 막은 걸 푼 뒤 10번째 메시지의 전송 응답을 받자마자 JVM을 강제 종료했습니다.

시점	결과
다운 직후	50건 모두 여전히 미발행
재기동 후	50건 모두 발행 완료
Kafka 토픽	레코드 60개, 서로 다른 eventId 50개 → 유실 0
중복	두 번 나간 eventId 10개, 두 레코드의 내용은 완전히 같음

실험: 실행 도구 · 결과 · Kafka 레코드 덤프

결론

발행하는 쪽은 **exactly-once**가 아니라 **at-least-once**였습니다. 중복의 폭은 한 묶음 안에서 다운되기 전까지 보낸 건수, 최대 100건입니다. 중복된 레코드는 eventId가 같으므로, 받는 쪽이 eventId로 걸러 내면 결과는 한 번 처리한 것과 같아집니다. Game이 받는 쪽일 때도 같은 원칙을 씁니다. 이미 처리한 이벤트 10건을 다시 흘려보냈을 때 추가 보상은 0건이었습니다.

2-3. 유실 0은 공짜로 만들어지는 게 아니다

두 실험 모두 유실은 0이었습니다. 하지만 대가가 있었습니다.

- 브로커 과반이 죽으면 브로커가 돌아올 때까지 아무것도 처리하지 않습니다.
- 전송 도중 서비스가 다운되면 같은 메시지가 최대 한 묶음만큼 두 번 나갑니다.

그리고 한 가지가 더 마음에 걸렸습니다. 파이프라인 전체가 멈춰 있는 동안 울린 알림이 하나도 없었습니다. 서비스는 살아 있었고 에러도 없었으니, Game 입장에서는 이벤트가 안 들어오는 조용한 새벽 구간과 구분할 수 없었던 것입니다.

그렇다면 다른 장애에서는 알림이 제대로 울릴까요? 하나씩 직접 멈춰 보기로 했습니다.

3. 멈추면 정말 알 수 있을까?

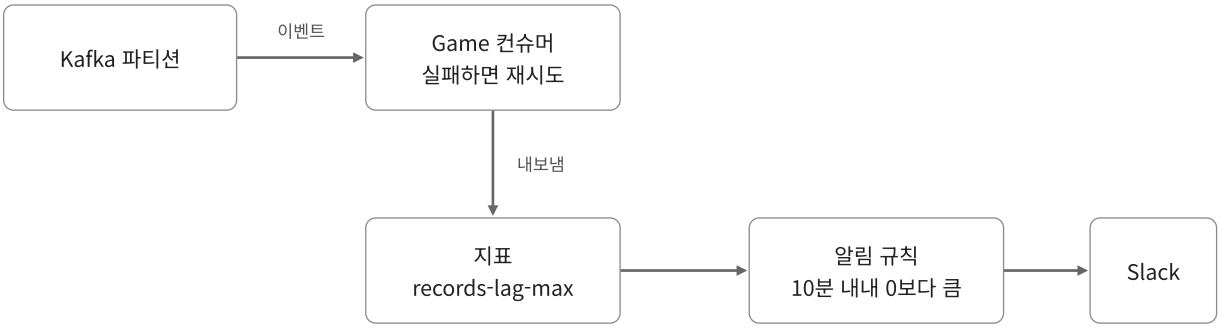
3-1. 알림은 울리는가 (지표)

① 20분 멈췄는데 lag은 0

설계상 보장

설계상 보장: 밀린 양이 10분 넘게 안 줄면 알림

한 이벤트가 실패를 거듭하면 해당 파티션의 뒤 이벤트들이 대기한다. 그 밀린 양(lag)을 지표로 본다



목적: 정각 스파이크처럼 이벤트가 한꺼번에 들어올 때를 제외하고, 이벤트가 한산한 새벽에 쌓인 lag가 계속 남아 있을 경우에 Slack으로 알림을 보낸다.

Game은 실패한 이벤트를 바로 버리지 않습니다. DB 연결이 끊긴 것처럼 기다리면 풀리는 장애는 성공할 때까지 재시도합니다. 한 이벤트가 실패를 거듭하면 해당 파티션의 뒤 이벤트들이 대기합니다.

그래서 **밀린 양(consumer lag)이 10분 내내 0으로 내려오지 않으면** 울리는 알림을 넣었습니다. 정각 스파이크처럼 이벤트가 한꺼번에 들어올 때는 제외하고, 이벤트가 한산한 새벽에 쌓인 lag가 계속 남아 있으면 Slack으로 알림을 보내기 위해서였습니다. 지표는 Kafka 클라이언트가 기본으로 내는 records-lag-max를 썼습니다.

실제 장애

실제 장애: 20분 멈췄는데 lag은 0

실패한 레코드로 되감을 때마다 뒤 레코드까지 다시 가져와서, **가져온 위치**는 늘 끝에 닿아 있었다



예전 지표 · 가져온 위치 기준
끝 − 가져온 위치 = 0
20분 내내 0 → 알림 안 울림

새 지표 · 커밋 기준 (30초마다 직접 잼)
끝 − 처리 끝낸 위치 = 6
20분 내내 6 → 603초 만에 Slack

같은 유저의 이벤트 6건이 한 파티션에서 계속 실패하도록 만들어, 파티션을 20분 동안 멈췄습니다. 첫 이벤트는 멈춤을 풀어 준 순간에야 처리됐습니다. 그런데 지표는 20분 내내 0이었고, 알림은 울리지 않았습니다.

records-lag-max는 **처리를 끝낸 위치**가 아니라 **가져온 위치** 기준이었습니다. 실패한 레코드로 되감을 때마다 컨슈머는 그 뒤 레코드까지 한 번에 다시 가져오고, 가져온 위치는 파티션 끝에 닿아 lag이 0으로 계산됩니다. 밀린 양이 한 번에 가져오는 양(파티션당 최대 1MB, 작은 이벤트면 수천 건)보다 적으면 멈춘 파티션이 보이지 않습니다. 이 알림을 넣은 이유였던 **이벤트가 한산할 때**가 정확히 그 조건이었습니다.

서비스가 30초마다 컨슈머 그룹의 커밋 오프셋과 파티션 끝 오프셋을 직접 읽어, 그 차이를 지표로 내게 바꿨습니다. 실패한 레코드는 커밋되지 않으므로, 멈춘 동안 이 값은 0보다 크게 남습니다. 같은 지표를 내 주는 별도 컨테이너(kafka-exporter)도 있었지만, 운영 서버 메모리를 생각해 서비스 안에서 재는 쪽을 택했습니다.

같은 멈춤을 다시 만들자 새 지표는 20분 내내 6, 기존 지표는 0이었습니다. 알림은 603초 만에 Slack에 도착했고, 멈춤을 풀자 해소됐습니다. 평상시 트래픽을 30분 흘렸을 때 오탐은 없었고, 처리량이 유입을 못 따라가는 상황에서도 같은 알림이 울렸습니다.

실험: 실행 도구 · 결과(두 지표 비교) · 결과(알림 603초) · lag 그래프 · 평상시 오탐 확인

결론

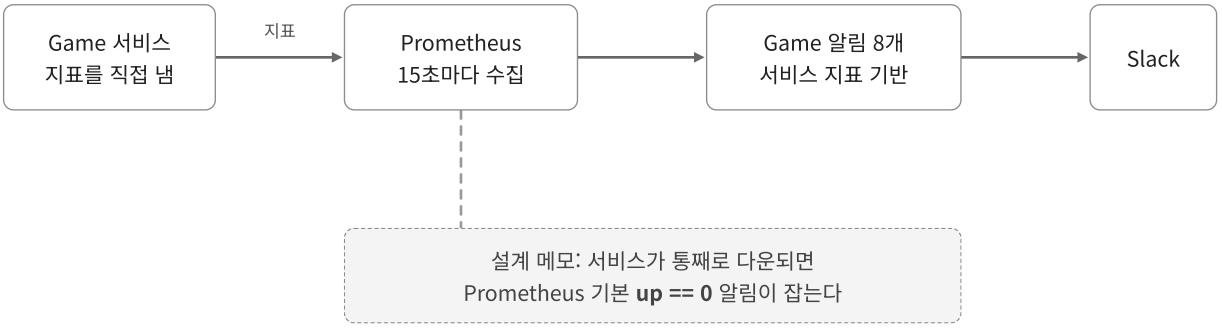
지표 이름이 lag이라고 해서 내가 원하는 lag은 아니었습니다. 알림은 넣는 것으로 끝나지 않고, 그 알림이 잡아야 할 상황을 실제로 만들어 봐야 합니다.

② 서비스가 다운되면 지표도 사라진다

설계상 보장

설계상 보장: 서비스가 다운되면 기본 알림이 잡는다

서비스가 내는 지표로 이상을 잡고, 서비스 자체가 다운되는 건 Prometheus 기본 알림이 잡을 거라고 예상했다

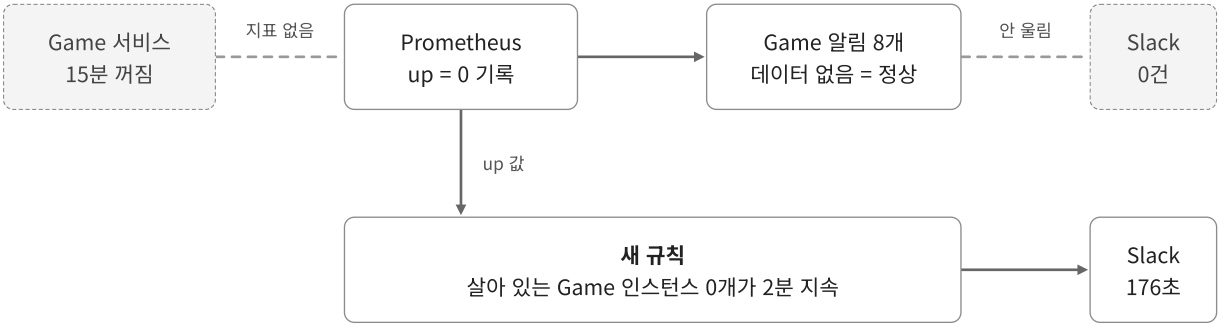


서비스가 내는 지표로 이상을 잡고, 서비스 자체가 다운되는 건 Prometheus 기본 `up == 0` 알림이 잡을 거라고 예상했습니다.

실제 장애

실제 장애: 서비스가 다운되자 지표도, 알림도 오지 않았다

그런 기본 규칙은 없었다. 서비스를 15분 끄자 456건이 밀렸는데, 규칙은 **데이터 없음 = 정상**으로 판정했다



하나라도 0이 아니라 **살아 있는 게 0**으로 건 이유: 블루/그린 배포에서 쉬는 쪽은 늘 up = 0이다.

그런 규칙은 없었습니다. 서비스를 15분 동안 껐더니 이벤트 456건이 밀렸는데, Game 알림 8개는 모두 조용했고 Slack에는 아무것도 오지 않았습니다.

Prometheus는 `up`을 0으로 정확히 기록하고 있었지만, 그 값을 보는 규칙이 없었습니다. 알림 8개는 모두 Game이 직접 내는 숫자를 보고 있었고, 서비스가 다운되면 그 숫자가 통째로 사라집니다. 규칙은 **데이터 없음 = 정상**으로 설정돼 있었습니다. 블루/그린 배포에서 쉬고 있는 쪽은 늘 데이터가 없기 때문에 일부러 그렇게 둔 설정이었습니다.

`up`을 보는 규칙을 새로 만들었습니다. **살아 있는 Game 인스턴스가 하나도 없는 상태가 2분 지속**되면 울립니다. **하나라도 0**으로 걸면 쉬는 쪽 때문에 알림이 늘 울리고, 2분은 배포나 재시작 때의 짧은 공백을 거르기 위한 값입니다. 기존 규칙을 **데이터 없음 = 알림**으로 바꾸는 방법도 있었지만, 쉬는 쪽 때문에 같은 문제가 생겨 버렸습니다.

다시 서비스를 끄자 176초 만에 Slack이 왔고, 그 순간에도 기존 알림 8개는 여전히 조용했습니다. 다운을 잡은 건 이 규칙 하나였습니다.

실험: 실행 도구 · 결과(다운 중 알림 0) · 결과(새 규칙 176초) · Slack 알림 화면

결론

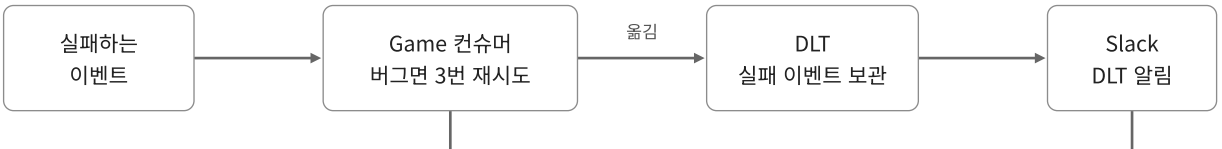
감시 대상이 스스로 숫자를 보내야 감시가 되는 구조라면, 대상이 다운되는 순간 감시도 함께 사라집니다. 바깥에서 두드려 보는 신호가 적어도 하나는 있어야 합니다.

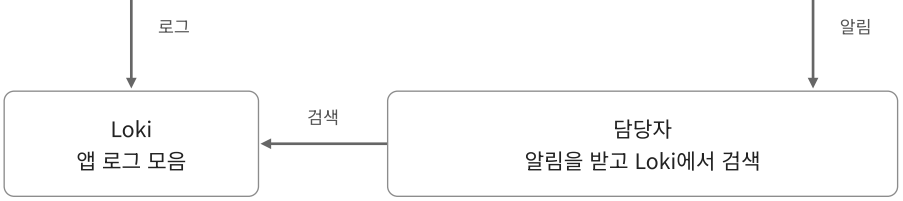
3-2. 왜 멈췄는지 아는가 (로그)

설계상 보장

설계상 보장: 실패는 DLT로, 원인은 로그에서

재시도해도 안 되는 이벤트는 DLT로 옮기고 알림을 보낸다. 앱 로그는 Loki에 모이니 원인은 거기서 찾는다.



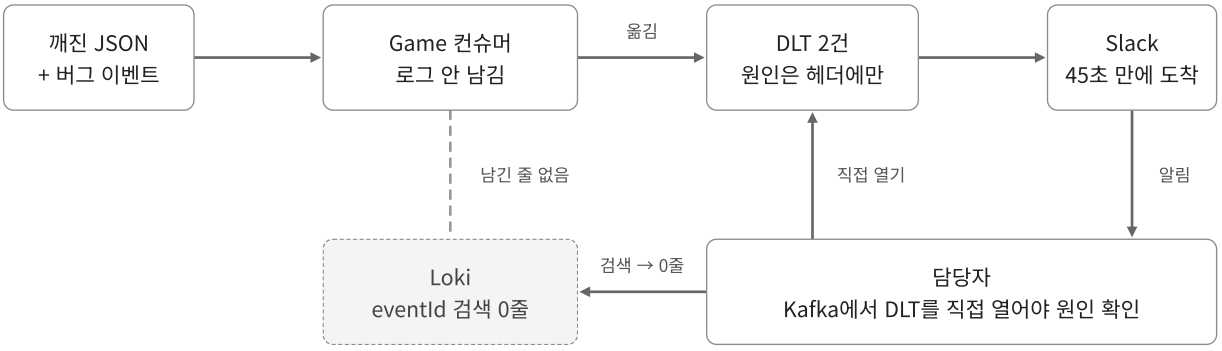


재시도해도 안 되는 이벤트는 DLT(Dead Letter Topic)로 옮깁니다. 형식이 깨진 이벤트는 바로, 코드 버그로 실패하는 이벤트는 세 번 재시도한 뒤 옮기고, DLT에 쌓이면 알림이 울립니다. 서비스 로그는 Loki에 모이니 원인은 거기서 찾는다고 생각했습니다.

실제 장애

실제 장애: 알림은 왔는데 로그는 0줄

깨진 JSON 1건과 버그 이벤트 1건을 넣었다. DLT에 2건이 쌓였고 Slack은 45초 만에 왔다



고친 뒤 · DLT로 옮길 때 ERROR 한 줄(위치 · eventId · 근본 원인), 재시도마다 WARN 한 줄
eventId 하나로 검색하면 깨진 JSON은 2줄, 버그 이벤트는 5줄

깨진 JSON 1건과 코드 버그로 실패하는 이벤트 1건을 넣었습니다. DLT에 2건이 쌓였고, 45초 만에 Slack 알림이 왔습니다.

그런데 Loki에서 두 이벤트를 찾으니 eventId, 유저 id, 예외 이름 모두 0줄이었습니다. 서비스의 원본 로그 파일에도 없었습니다. 수집 문제가 아니라 서비스가 아예 남기지 않은 것이었습니다. 원인은 DLT 레코드의 헤더에만 있었고, 알림을 받은 사람은 Kafka를 직접 열어 봐야 원인을 알 수 있었습니다.

DLT로 보낼 때 알림용 카운터를 세려고 처리기를 직접 만들면서, 스프링 기본 처리기가 남기던 로그가 함께 사라졌던 것입니다. 테스트는 헤더와 오프셋, 카운터만 확인했고 로그는 보지 않았습니다.

두 곳에 한 줄씩 남기게 고쳤습니다.

- DLT로 옮긴 직후: 원래 위치, 유저 id, eventId, 근본 원인을 ERROR 한 줄로
- 재시도할 때마다: 몇 번째 시도인지, 어떤 종류의 실패인지를 WARN 한 줄로

eventId는 깨진 JSON에서도 건질 수 있게 파싱 대신 정규식으로 꺼냈습니다. 스택 트레이스는 넣지 않았습니다. 여러 줄이 되면 Loki의 줄 단위 검색이 깨지고, 이미 DLT 헤더에 들어 있기 때문입니다.

같은 실험을 다시 돌리자 eventId 하나로 그 이벤트의 이력이 모두 나왔습니다. 깨진 JSON은 **형식 위반** WARN 한 줄과 DLT ERROR 한 줄(근본 원인 JsonParseException, 열 위치까지), 버그 이벤트는 1 · 2 · 4초 간격의 재시도 WARN 네 줄과 DLT ERROR 한 줄이 나왔습니다.

실험: 실행 도구 · 결과(고치기 전, 0줄) · 결과(고친 뒤) · Loki 로그 화면

결론

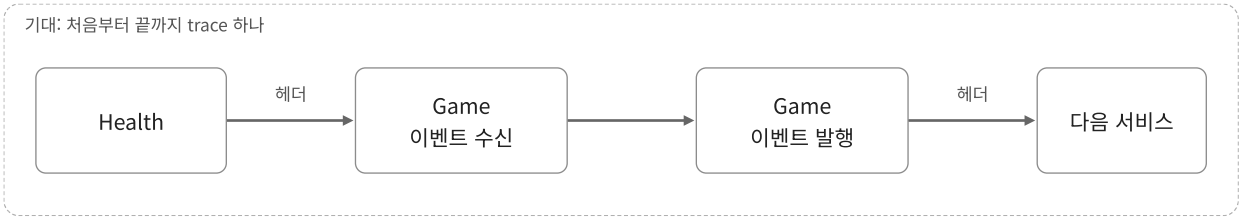
알림은 **무슨 일이 생겼다**까지만 알려 줍니다. **어느 이벤트가 왜**는 로그의 몫이고, 알림이 울리는 것과 원인을 찾을 수 있는 것은 따로 확인해야 합니다.

3-3. 어디까지 갔는지 아는가 (트레이스)

설계상 보장

설계상 보장: 헤더로 이어 받으니 trace는 하나

Kafka 메시지는 traceparent 헤더로 trace를 이어 받는다. 서비스를 건너도 한 trace로 보일 거라고 생각했다



서비스에는 Zipkin 분산 추적이 붙어 있고, Kafka 메시지는 헤더(traceparent)로 trace를 이어 받습니다. 그러니 Health가 보낸 이벤트가 Game을 거쳐 다음 서비스로 나가는 흐름이 한 trace로 보일 거라고 생각했습니다.

실제로는

실제: Outbox에서 trace가 끊긴다

발행은 나중에, 다른 스레드 · 다른 트랜잭션에서 일어나서 원래 trace 정보가 남아 있지 않다

고치기 전 (구조상)



고친 뒤 (실험으로 확인)



실험: traceId를 고정한 헤더로 이벤트를 넣자 수신 → Outbox 발행 → Kafka 전송이 Zipkin에서 한 trace로 이어졌고, 같은 traceId로 Loki 로그 11줄이 나왔다.

Game의 출구는 Outbox입니다. 이벤트를 받은 트랜잭션은 Outbox에 행을 저장하고 끝나고, 실제 발행은 나중에 스케줄러 스레드가 다른 트랜잭션에서 합니다. 발행하는 시점에는 원래 trace 정보가 남아 있지 않아 새 trace로 시작하게 되고, Game의 입구와 출구가 끊깁니다.

Outbox에 행을 저장할 때 현재 trace 정보를 같이 저장하고, 발행할 때 그 값을 꺼내 부모로 되살리게 고쳤습니다. 샘플링도 부모의 결정을 따르므로, 샘플링 비율을 낮춰도 뽑힌 trace는 끝까지 이어집니다.

Health 대신 traceId를 고정한 헤더를 붙여 이벤트를 넣었습니다. Zipkin에서 한 trace 안에 **이벤트 수신 → Outbox 발행(다른 스레드, 다른 트랜잭션) → Kafka 전송**이 한 줄로 이어졌고, 밖으로 나가는 메시지에도 같은 traceId가 실렸습니다. 다음 서비스는 이 값을 이어 받으면 됩니다.

실험: 실행 도구 · 결과

결론

> Outbox처럼 처리를 나중으로 미루는 경계는 trace가 끊기는 자리입니다. trace 정보도 데이터와 함께 저장해야 이어집니다.

3-4. 세 신호를 잇기

이제 장애가 나면 이 순서로 따라갈 수 있습니다. 알림이 울리면 DLT 로그에서 eventId와 원인을 찾고, 그 로그 줄에 붙은 traceId로 Zipkin에서 이벤트가 어디까지 갔는지 봅니다. 실제로 같은 traceId로 Loki를 검색하자, 이벤트를 받은 스레드와 발행한 스레드의 로그 11줄이 한꺼번에 나왔습니다.

하지만 2장에서 본 브로커 멈춤은 이 셋 중 어느 것으로도 잡히지 않습니다. 서비스는 살아 있었고, 실패한 이벤트도, 남을 로그도 없었기 때문입니다.

---

4. 에필로그

감시도 유실을 막는 장치다

파티션이 20분 멈췄을 때도, 서비스가 15분 동안 다운됐을 때도 이벤트는 Kafka에 그대로 남아 있었습니다. 하지만 그동안 아무도 몰랐습니다. Kafka는 이벤트를 보관 기간만큼만 들고 있기 때문에, 멈춘 걸 모른 채 그 기간이 지나면 그때는 정말로 유실됩니다. 그래서 저는 알림과 로그, 트레이스도 유실을 막는 장치라고 생각합니다.

보장과 검증은 다르다

Kafka 복제, Outbox, lag 알림, DLT 모두 설계상으로는 맞았습니다. 하지만 직접 꺼 보기 전에는 2대가 죽으면 읽기까지 멈춘다는 것도, 중복이 한 묶음만큼 생긴다는 것도, lag 알림이 멈춘 파티션을 못 본다는 것도 몰랐습니다. 설계상 보장은 **그래야 한다**이고, 검증은 **실제로 그랬다**입니다.

아직 남은 허점

- 브로커 과반이 죽어 파이프라인 전체가 멈춘 동안 울린 알림이 없습니다. 서비스가 내는 지표로는 이 멈춤이 보이지 않아, 브로커 쪽 지표(복제본이 부족한 파티션 수 등)를 따로 봐야 합니다.

- 브로커 2대가 죽었을 때 읽기가 멈춘 원인은 high watermark로 보고 있지만, 아직 그 값을 직접 확인하지는 못했습니다.